



Southeast University

Department of Computer Science and Engineering (CSE)

School of Sciences and Engineering

Semester: (Summer, Year: 2026)

LAB REPORT NO: 05

Course Title: Operating Systems Lab

Course Code: CSE362.2

Batch: 65

Lab Experiment Name: Implementing non preemptive CPU scheduling algorithms.

Student Details

Name		ID
1.	Md. Mahmud Hossain	2023200000799

Course Teacher's Name : Md. Ismail

Designation : Lecturer

Submission Date : 24-08-26

Lab Report Status

Marks:

Comments:.....

Signature:.....

Date:.....

Introduction:

CPU scheduling is a core operating system function that determines which process in the ready queue is allocated the CPU for execution. In non-preemptive scheduling, once a process is allocated the CPU, it holds the processor until it either terminates or requests an I/O operation. This lab report focuses on three standard non-preemptive CPU scheduling algorithms: First-Come, First-Served (FCFS), Shortest Job First (SJF), and Priority Scheduling. Evaluation metrics include Completion Time (\$CT\$), Turnaround Time (\$TAT\$), Waiting Time (\$WT\$), total CPU Idle Time, and overall CPU Utilization.

Objectives:

- To understand the working principles of non-preemptive CPU scheduling algorithms.
- To implement FCFS, SJF (Non-preemptive), and Priority (Non-preemptive) scheduling algorithms in C.
- To dynamically take user input for process details (Arrival Time, Burst Time, Priority).
- To compute and display individual process metrics (\$WT\$, \$TAT\$) and aggregate system metrics (Average \$WT\$, Average \$TAT\$, Idle Time, CPU Utilization).

Implementation 1:

Implementing FCFS scheduling algorithm

C code:

```
#include <stdio.h>
#include <stdbool.h>

void sort_by_arrival(int pid[], int AT[], int BT[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (AT[j] > AT[j + 1]) {
                int temp = AT[j]; AT[j] = AT[j + 1]; AT[j + 1] = temp;
                temp = BT[j]; BT[j] = BT[j + 1]; BT[j + 1] = temp;
                temp = pid[j]; pid[j] = pid[j + 1]; pid[j + 1] = temp;
            }
        }
    }
}

int main() {
    int n;
    printf("Enter number of processes: ");
```

```

scanf("%d", &n);

int pid[n], AT[n], BT[n], CT[n], TAT[n], WT[n];

for (int i = 0; i < n; i++) {
    printf("Process %d - Enter PID, Arrival Time, Burst Time: ", i + 1);
    scanf("%d %d %d", &pid[i], &AT[i], &BT[i]);
}

sort_by_arrival(pid, AT, BT, n);

int current_time = 0, idle_time = 0;
float total_tat = 0, total_wt = 0;

for (int i = 0; i < n; i++) {
    if (current_time < AT[i]) {
        idle_time += (AT[i] - current_time);
        current_time = AT[i];
    }
    CT[i] = current_time + BT[i];
    current_time = CT[i];
    TAT[i] = CT[i] - AT[i];
    WT[i] = TAT[i] - BT[i];

    total_tat += TAT[i];
    total_wt += WT[i];
}

double cpu_utilization = ((double)(current_time - idle_time) / current_time) * 100;

printf("\n--- FCFS Scheduling Result ---\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], AT[i], BT[i], CT[i], TAT[i], WT[i]);
}

printf("\nAverage Waiting Time: %.2f", total_wt / n);
printf("\nAverage Turnaround Time: %.2f", total_tat / n);
printf("\nTotal Idle Time: %d units", idle_time);
printf("\nCPU Utilization: %.2f%%\n", cpu_utilization);

```

```
    return 0;
}
```

Screenshot:

```
mahmud@mahmud-VMware-Virtual-Platform:~$ gcc fcpc.c -o my_obj
mahmud@mahmud-VMware-Virtual-Platform:~$ ./my_obj
Enter number of processes: 3
Process 1 - Enter PID, Arrival Time, Burst Time: 1 0 4
Process 2 - Enter PID, Arrival Time, Burst Time: 2 1 3
Process 3 - Enter PID, Arrival Time, Burst Time: 3 2 1

--- FCFS Scheduling Result ---
PID      AT      BT      CT      TAT      WT
1         0       4       4       4       0
2         1       3       7       6       3
3         2       1       8       6       5

Average Waiting Time: 2.67
Average Turnaround Time: 5.33
Total Idle Time: 0 units
CPU Utilization: 100.00%
mahmud@mahmud-VMware-Virtual-Platform:~$
```

Implementation 2:

Implementing SJF scheduling algorithm

C code:

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n], AT[n], BT[n], CT[n], TAT[n], WT[n];
    bool completed[n];

    for (int i = 0; i < n; i++) {
```

```

printf("Process %d - Enter PID, Arrival Time, Burst Time: ", i + 1);
scanf("%d %d %d", &pid[i], &AT[i], &BT[i]);
completed[i] = false;
}

int current_time = 0, completed_count = 0, idle_time = 0;
float total_tat = 0, total_wt = 0;

while (completed_count < n) {
    int idx = -1;
    int min_bt = 1e9;

    for (int i = 0; i < n; i++) {
        if (!completed[i] && AT[i] <= current_time) {
            if (BT[i] < min_bt) {
                min_bt = BT[i];
                idx = i;
            }
        }
    }

    if (idx != -1) {
        CT[idx] = current_time + BT[idx];
        current_time = CT[idx];
        TAT[idx] = CT[idx] - AT[idx];
        WT[idx] = TAT[idx] - BT[idx];

        total_tat += TAT[idx];
        total_wt += WT[idx];
        completed[idx] = true;
        completed_count++;
    } else {
        current_time++;
        idle_time++;
    }
}

double cpu_utilization = ((double)(current_time - idle_time) / current_time) * 100;

printf("\n--- Non-Preemptive SJF Scheduling Result ---\n");

```

```

printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], AT[i], BT[i], CT[i], TAT[i], WT[i]);
}

printf("\nAverage Waiting Time: %.2f", total_wt / n);
printf("\nAverage Turnaround Time: %.2f", total_tat / n);
printf("\nTotal Idle Time: %d units", idle_time);
printf("\nCPU Utilization: %.2f%%\n", cpu_utilization);

return 0;
}

```

Screenshot:

```

mahmud@mahmud-VMware-Virtual-Platform:~$ nano sjf.c
mahmud@mahmud-VMware-Virtual-Platform:~$ gcc sjf.c -o my_obj
mahmud@mahmud-VMware-Virtual-Platform:~$ ./my_obj
Enter number of processes: 3
Process 1 - Enter PID, Arrival Time, Burst Time: 1 0 7
Process 2 - Enter PID, Arrival Time, Burst Time: 2 2 4
Process 3 - Enter PID, Arrival Time, Burst Time: 3 4 1

--- Non-Preemptive SJF Scheduling Result ---
PID      AT      BT      CT      TAT      WT
1         0       7       7       7       0
2         2       4      12      10       6
3         4       1       8       4       3

Average Waiting Time: 3.00
Average Turnaround Time: 7.00
Total Idle Time: 0 units
CPU Utilization: 100.00%
mahmud@mahmud-VMware-Virtual-Platform:~$ █

```

Implementation 3:

Implementing priority scheduling algorithm

C code:

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    // Note: Lower priority integer value represents HIGHER priority
    int pid[n], AT[n], BT[n], priority[n], CT[n], TAT[n], WT[n];
    bool completed[n];

    for (int i = 0; i < n; i++) {
        printf("Process %d - Enter PID, Arrival Time, Burst Time, Priority: ", i + 1);
        scanf("%d %d %d %d", &pid[i], &AT[i], &BT[i], &priority[i]);
        completed[i] = false;
    }

    int current_time = 0, completed_count = 0, idle_time = 0;
    float total_tat = 0, total_wt = 0;

    while (completed_count < n) {
        int idx = -1;
        int highest_priority = 1e9;

        for (int i = 0; i < n; i++) {
            if (!completed[i] && AT[i] <= current_time) {
                if (priority[i] < highest_priority) {
                    highest_priority = priority[i];
                    idx = i;
                }
            }
        }

        if (idx != -1) {
            CT[idx] = current_time + BT[idx];
```

```

    current_time = CT[idx];
    TAT[idx] = CT[idx] - AT[idx];
    WT[idx] = TAT[idx] - BT[idx];

    total_tat += TAT[idx];
    total_wt += WT[idx];
    completed[idx] = true;
    completed_count++;
} else {
    current_time++;
    idle_time++;
}
}

double cpu_utilization = ((double)(current_time - idle_time) / current_time) * 100;

printf("\n--- Non-Preemptive Priority Scheduling Result ---\n");
printf("PID\tAT\tBT\tPri\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], AT[i], BT[i], priority[i], CT[i], TAT[i],
WT[i]);
}

printf("\nAverage Waiting Time: %.2f", total_wt / n);
printf("\nAverage Turnaround Time: %.2f", total_tat / n);
printf("\nTotal Idle Time: %d units", idle_time);
printf("\nCPU Utilization: %.2f%%\n", cpu_utilization);

return 0;
}

```

Screenshot:

```
mahmud@mahmud-VMware-Virtual-Platform:~$ nano priority.c
mahmud@mahmud-VMware-Virtual-Platform:~$ gcc priority.c -o my_obj
mahmud@mahmud-VMware-Virtual-Platform:~$ ./my_obj
Enter number of processes: 3
Process 1 - Enter PID, Arrival Time, Burst Time, Priority: 1 0 4 2
Process 2 - Enter PID, Arrival Time, Burst Time, Priority: 2 1 3 1
Process 3 - Enter PID, Arrival Time, Burst Time, Priority: 3 2 1 3

--- Non-Preemptive Priority Scheduling Result ---
PID      AT      BT      Pri      CT      TAT      WT
1         0        4        2        4        4        0
2         1        3        1        7        6        3
3         2        1        3        8        6        5

Average Waiting Time: 2.67
Average Turnaround Time: 5.33
Total Idle Time: 0 units
CPU Utilization: 100.00%
mahmud@mahmud-VMware-Virtual-Platform:~$
```

Conclusion:

In this lab experiment, non-preemptive CPU scheduling algorithms—FCFS, Shortest Job First (SJF), and Priority Scheduling—were implemented and evaluated in C. FCFS schedules processes strictly by arrival order, which can cause high waiting times (convoy effect). SJF minimizes overall waiting times by selecting the process with the shortest burst time among available processes. Priority scheduling executes processes based on assigned priority criteria. Calculating CPU idle time and CPU utilization demonstrated how efficiently the processor is utilized under varying workloads and scheduling strategies.